

Analytics Engineering: Session 17

Materializations, Incremental Models & Snapshots

Goal

Understand when to use each materialization and how to capture historical changes.

By the end of this session you will have:

1. A mental model for choosing view, table, ephemeral, or incremental
2. `mart_daily_sessions` — an incremental model aggregating website traffic by day
3. `customer_snapshot` — a check-strategy snapshot tracking SCD2 customer history
4. A live demo showing how `dbt snapshot` captures a real attribute change

Checklist

- [] `models/marts/mart_daily_sessions.sql` — incremental model on `stg_website_sessions`
- [] `snapshots/customer_snapshot.sql` — `check`-strategy snapshot on `source('raw', 'customers')`
- [] `dbt build -s mart_daily_sessions` passes on first and second run
- [] `dbt snapshot` captures a live attribute change — two rows for the mutated customer

Materializations — the four options

view

Re-runs SQL on every query
Always fresh · no storage
cost
Slow on large datasets

table

Full rebuild on each run
Fast queries · stale until
run
Expensive on large datasets

ephemeral

Compiles to an inline CTE
No storage · no direct
queries
Zero overhead at query
time

incremental

Appends / merges new
rows only
Fastest builds on large
tables
Requires unique_key

View

The default for `staging/`.dbt runs the SQL as a query every time another model `ref()`s it — nothing is written to the warehouse.

- No storage cost, always reflects the latest source data
- Slow on large datasets — every downstream query re-runs the SQL

```
{{ config(materialized='view') }}
```

Table

The default for `intermediate/` and `marts/` .dbt drops and recreates the table on every `dbt run` .

- Fast queries — data is materialized to the warehouse
- Stale until the next run
- Expensive on large tables — full rebuild every time, even if nothing changed

```
{{ config(materialized='table') }}
```

Ephemeral

No database object is created. dbt inlines the model's SQL as a CTE inside every model that `ref()` s it.

- Zero storage cost — no warehouse object
- **Cannot** be queried directly from a database client
- Useful for reusable logic that should not be a standalone table

```
{{ config(materialized='ephemeral') }}
```

Overriding the project default

The project sets a default materialization for every layer in `dbt_project.yml`:

```
models:
  dbt_ie:
    marts:
      materialized: table  # applies to all marts
```

A `{{ config() }}` block at the top of a model overrides it:

```
{{ config(
  materialized='incremental',
  unique_key='session_day'
) }}
```

Model-level `config()` always wins over the project default.

Why incremental?

Every `dbt run` rebuilds every model from scratch by default.

For a table with **100M rows**, a full rebuild processes all 100M rows — even if only 50K new rows arrived yesterday.

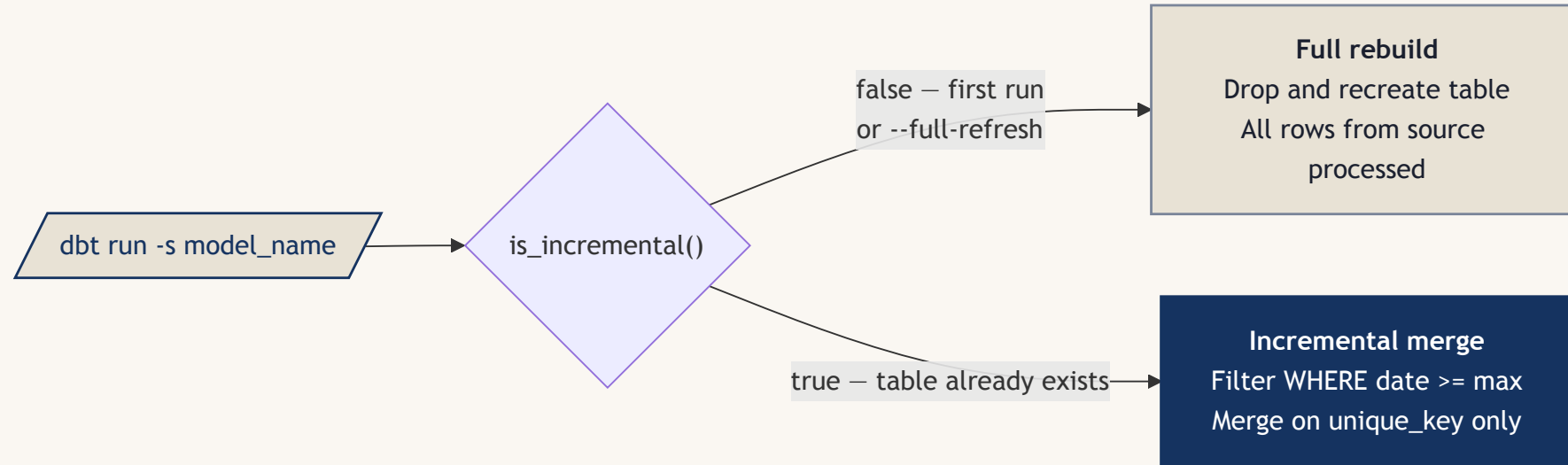
Incremental models solve this:

- **First run** — `is_incremental()` returns false: full table built from scratch
- **Subsequent runs** — `is_incremental()` returns true: only new or updated rows processed

Result: build time drops from minutes to seconds on large tables.

This dataset is teaching-sized — but the pattern scales to any volume.

is_incremental()



The model in context



The dashed arrow shows `{{ this }}` feeding a filter back into the staging step — only when the table already exists.

`{{ this }}` — how the filter is built

```
{{ config(materialized='incremental', unique_key='session_day') }}

with sessions as (
    select
        cast(session_date as date) as session_day,
        converted,
        source
    from {{ ref('stg_website_sessions') }}
    {% if is_incremental() %}
        -- {{ this }} resolves to the schema + table at compile time
        where cast(session_date as date) > (select max(session_day) from {{ this }})
    {% endif %}
)
...
```

`{{ this }}` is a Jinja reference that dbt resolves to the model's own existing table. Run

Incremental config keys

Key	Purpose
<code>materialized</code>	Set to <code>'incremental'</code>
<code>unique_key</code>	Column(s) to merge on — prevents duplicate rows
<code>incremental_strategy</code>	<code>append</code> / <code>merge</code> / <code>delete+insert</code> (DuckDB default: <code>merge</code>)
<code>on_schema_change</code>	What to do if the model's columns change

Incremental strategies

Strategy	What it does	Best for
<code>append</code>	Insert new rows only — no deduplication	Immutable event streams
<code>merge</code>	Upsert on <code>unique_key</code> — updates existing rows	Most use cases
<code>delete+insert</code>	Delete matching rows, then re-insert	Partitioned tables

Default on DuckDB: `merge`. Requires `unique_key`.

--full-refresh and on_schema_change

Force a full rebuild at any time:

```
dbt run -s mart_daily_sessions --full-refresh
```

`on_schema_change` controls what happens when the model's column list changes:

Value	Behavior
<code>ignore</code> (default)	Silently ignore new columns
<code>append_new_columns</code>	Add new columns to the existing table
<code>sync_all_columns</code>	Add and remove columns to match the model
<code>fail</code>	Raise an error — force a deliberate decision

Three scenarios at a glance

Scenario	Command	<code>is_incremental()</code>	What happens
First run	<code>dbt run -s mart_daily_sessions</code>	<code>false</code> — table doesn't exist	Full source scan · table created
Subsequent run	<code>dbt run -s mart_daily_sessions</code>	<code>true</code> — table exists	Filtered scan · rows merged on <code>session_day</code>
Force rebuild	<code>dbt run -s mart_daily_sessions --full-refresh</code>	<code>false</code> — flag overrides	Full source scan · table dropped and recreated

The `--full-refresh` flag is the escape hatch: it tells dbt to treat the model as if it's the first run, regardless of whether the table exists.

Real data: dates are strings

Source date columns in this dataset are `VARCHAR`, not `DATE`.

```
-- ✗ type mismatch – session_date is a VARCHAR
where session_date > (select max(session_day) from {{ this }})

-- ✓ cast before comparing
where cast(session_date as date) > (select max(session_day) from {{ this }})
```

Always cast before using a string date in an incremental predicate.

Exercise 1— mart_daily_sessions (20 min)

Create `models/marts/mart_daily_sessions.sql` using `{{ ref('stg_website_sessions') }}`.

Configure:

- `materialized='incremental'`
- `unique_key='session_day'`
- `on_schema_change='append_new_columns'`

Add an `{% if is_incremental() %}` filter using `cast(session_date as date)`.

Exercise 1 — output columns (cont.)

Column	Description
<code>session_day</code>	<code>session_date</code> cast to <code>date</code>
<code>sessions</code>	Total sessions
<code>conversions</code>	Sessions where <code>converted = true</code>
<code>conversion_rate</code>	<code>conversions / sessions * 100</code> , rounded to 2 d.p.
<code>unique_sources</code>	Distinct values of <code>source</code>

Verify — three scenarios

① **First run** — `is_incremental()` is false, full table built:

```
dbt build -s mart_daily_sessions
```

② **Subsequent run** — `is_incremental()` is true, only new rows merged:

```
dbt build -s mart_daily_sessions           # run again, same data
```

③ **Force rebuild** — drop and recreate from scratch:

```
dbt build -s mart_daily_sessions --full-refresh
```

After each run, inspect the compiled SQL to see the predicate change:

```
dbt compile -s mart_daily_sessions  
# check target/compiled/dbt ie/models/marts/mart_daily_sessions.sql
```

Snapshots — capturing history

Incremental models track **new rows**. But what if a customer's `customer_segment` changes from `regular` to `enterprise` ?

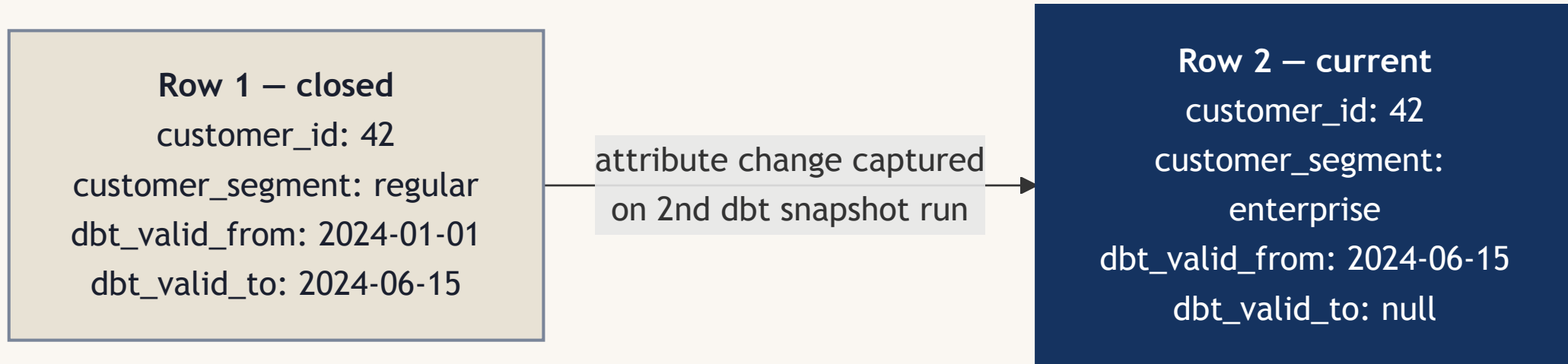
With `merge` on `customer_id`, dbt **overwrites** the old value — history is lost.

Snapshots solve this:

- On each `dbt snapshot` run, dbt compares the current source against the previous snapshot
- Changed rows get a new record; the old row is **closed** with a `dbt_valid_to` timestamp
- This pattern is called **Slowly Changing Dimension Type 2 (SCD2)**

Unlike models, snapshots live in `snapshots/` and run with `dbt snapshot` .

Snapshot anatomy



`dbt_valid_from` — when this row became active · `dbt_valid_to` — when it was closed
(`null` = current row)

Strategy: timestamp vs check

Strategy	How it detects changes	Requires
timestamp	Compares <code>updated_at</code> column between runs	A reliable <code>updated_at</code> in the source
check	Hashes the values in <code>check_cols</code> each run	A list of columns to watch

Our `customers` source has no `updated_at` → we use `check`.

`check_cols` accepts a list of column names or `'all'` to compare every column.

The {% snapshot %} block

```
{% snapshot customer_snapshot %}

{{ config(
    target_schema='snapshots',
    unique_key='customer_id',
    strategy='check',
    check_cols=['customer_segment', 'country']
)}}

select * from {{ source('raw', 'customers') }}

{% endsnapshot %}
```

File location: `snapshots/customer_snapshot.sql` · Run with: `dbt snapshot`

`select *` here is intentional — snapshots capture the full source state verbatim.

Exercise 2 — customer_snapshot (15 min)

1. Create `snapshots/customer_snapshot.sql` as shown above
2. Run `dbt snapshot` — confirm `snapshots.customer_snapshot` has one row per customer
3. Open a DuckDB session and update a customer's segment:

```
update main.customers
  set customer_segment = 'enterprise'
 where customer_id = 1;
```

4. Run `dbt snapshot` again
5. Query the snapshot and confirm two rows exist for `customer_id = 1`

Verify

```
dbt snapshot    # initial capture – one row per customer  
# update main.customers as above  
dbt snapshot    # captures the change
```

Query to confirm SCD2 captured the change:

```
select customer_id, customer_segment, dbt_valid_from, dbt_valid_to  
from snapshots.customer_snapshot  
where customer_id = 1  
order by dbt_valid_from;
```

To reset the database at any point: `python create_db.py`

Bonus

- Change `check_cols` to `'all'` in `customer_snapshot` — does behavior change?
- Update a column **not** in `check_cols` (e.g., `first_name`) and run `dbt snapshot` — confirm no new row is created
- Change the incremental strategy in `mart_daily_sessions` to `delete+insert` — compare the compiled SQL
- Add `on_schema_change='fail'` to `mart_daily_sessions`, add a new column, then run `dbt build`

What have we learned in this session

- **Materializations:** view, table, ephemeral trade-offs; model-level `{{ config() }}` overrides the project default
- **Incremental:** `is_incremental()`, `unique_key`, `merge` strategy, `--full-refresh`, `on_schema_change`, casting `VARCHAR` dates
- **Snapshots:** SCD2 concept, `check` strategy, `dbt_valid_from` / `dbt_valid_to`, `dbt snapshot` command

Next Session: Certification Preparation & Review